```
BottomTwo: @@A = top A, @@B = Two B, C = Two C
```

Because of this behavior, class variables tend to be confusing, hard to debug, and hard to reason about and we recommend avoiding them.

Instance variables are available within instance methods throughout a class body. Referencing an uninitialized instance variable returns nil. Each object (instance of a class) has a unique set of instance variables.

Local variables are unique in that their scopes are statically determined but their existence is established dynamically.

A local variable is created dynamically when it is first assigned a value during program execution. However, the scope of a local variable is statically determined to be the immediately enclosing block, method definition, class definition, module definition, or top-level program. Local variables with the same name are different variables if they appear in disjoint scopes.

Method parameters are considered to be variables local to that method.

Block parameters are assigned values when the block is invoked.

If a local variable is first assigned in a block, it is local to the block. If a block uses a variable that is previously defined in the scope containing the block's definition, then the block will share that variable with the scope.

There are two exceptions to this rule. Block parameters are always local to the block. In addition, variables listed after a semicolon at the end of the block parameter list are also always local to the block.

```
a = 1
b = 2
c = 3

some_method do |b; c|
  a = b + 1
  c = a + 1
  d = c + 1
end
```

In this previous example, the variable a inside the block is shared with the surrounding scope. The variables b and c are not shared, because they are listed in the block's parameter list, and the variable d is not shared because it occurs only inside the block.

A block takes on the set of local variables in existence at the time that it is created. This forms part of its binding. Note that although the binding of the variables is fixed at this point, the block will have access to the *current* values of these variables when it executes. The binding preserves these variables even if the original enclosing scope is destroyed.

The bodies of while, until, and for loops do not act as blocks. They are part of the scope that contains them; previously existing locals can be used in the loop, and any new locals created will be available outside the bodies afterward.